

Lab 1-2

This lab assignment covers Week 1 and 2 (Lectures 1 - 5):

1. Data
2. Linear Regression 1 (Correlation and Ordinary Least Squares)
3. Train Test Splitting
4. Linear Regression 2 (Multiple Regression and Regularization)
5. Classification Errors

Submission And Assignment Instructions

Submit your `Lab1-2_lastname_firstname.pdf` file on canvas with your answers clearly marked (✅) and code commented. Skeleton code and/or comments are provided throughout to help you.

Formatting Notes:

- ❓ Questions you must answer or tasks are marked with a "❓"
- ✅ Answers you should give are marked with "✅ Answer:"
- *Helpful hints are usually given in italics*
- Code you need to write is marked with `## ❓ YOUR CODE HERE`
 - ⭐ Bonus questions are marked with a "⭐"
 - 🍓 Bonus answers are marked with a "🍓"

Please don't erase ANY of these markings! They're to help the TAs find your answers and code as much as they're to help you find the questions.

Section 1: Math!

Answers may be submitted as Latex typeset equations OR inserted as pictures of your handwritten notes. For the later, please ensure you take a clear scan/picture of your readable work. If we can not read your writing, we might have to deduct points for incomplete answers.

1.1: Derivation of the Ordinary Least Squares Estimator with Calculus

In the lecture you were introduced to the OLS equations, let's find out where these equations come from!

❓ 1.1.1 The mean squared error (MSE) function for a simple linear model can be expressed as $E(\beta_0, \beta_1) = \frac{1}{n} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2$. Take the partial derivative of E wrt β_0 .

✓ Answer:

$$1.1.1 \quad \frac{\partial E}{\partial \beta_0} = -\frac{2}{n} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)$$

? 1.1.2 Take the partial derivative of E wrt β_1 .

✓ Answer:

$$1.1.2 \quad \frac{\partial E}{\partial \beta_1} = -\frac{2}{n} \sum_{i=1}^n x_i (y_i - \beta_0 - \beta_1 x_i)$$

? 1.1.3 Set both partial derivatives equal to 0 and solve for the optimal parameter values of β_0 and β_1 . You must show your work.

✓ Answer:

1.1.3

$$\begin{aligned} \frac{\partial E}{\partial \beta_0} = 0 &= -\frac{2}{n} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i) \\ &= \frac{1}{n} (n\bar{y} - n\beta_0 - n\beta_1 \bar{x}) \\ \beta_0 &= \bar{y} - \beta_1 \bar{x} \end{aligned}$$

$$\begin{aligned} \frac{\partial E}{\partial \beta_1} = 0 &= -\frac{2}{n} \sum_{i=1}^n x_i (y_i - \beta_0 - \beta_1 x_i) \\ &= -\frac{2}{n} \sum_{i=1}^n x_i [y_i - (\bar{y} - \beta_1 \bar{x}) - \beta_1 x_i] \\ &= \sum_{i=1}^n x_i [y_i - (\bar{y} - \beta_1 \bar{x}) - \beta_1 x_i] \\ &= \sum_{i=1}^n x_i y_i - \sum_{i=1}^n x_i \bar{y} + \sum_{i=1}^n x_i \beta_1 \bar{x} - \sum_{i=1}^n \beta_1 x_i^2 \\ &= \sum_{i=1}^n x_i y_i - n\bar{x}\bar{y} + n\beta_1 \bar{x}^2 - \sum_{i=1}^n \beta_1 x_i^2 \\ &= \sum_{i=1}^n x_i y_i - n\bar{x}\bar{y} - \beta_1 \left(-n\bar{x}^2 + \sum_{i=1}^n x_i^2 \right) \\ &= \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}) - \beta_1 \sum_{i=1}^n (x_i - \bar{x})^2 \\ \beta_1 \sum_{i=1}^n (x_i - \bar{x})^2 &= \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}) \\ \beta_1 &= \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2} \end{aligned}$$

★ 1.2: Bonus – Linear Algebra Notation for Multiple Linear Regression

We can rewrite the OLS equations above like this:

$$\hat{Y} = \begin{pmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_1 \end{pmatrix} \quad (1)$$

$$= \begin{pmatrix} 1 & | \\ \vdots & \vec{x} \\ 1 & | \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_1 \end{pmatrix} \quad (2)$$

$$= X\vec{\beta} \quad (3)$$

★ 1.2.1 Why do we need a column of 1s in the X matrix above? (one sentence) \

🍓 Answer: The column of 1s in the X matrix above ensures that the β_0 constant is added to the equation.

★ 1.2.2 Imagine we now have 2 independent variables, which we'll call $\vec{x}_1$ and $\vec{x}_2$, that we wish to use to predict $\vec{y}$. We will also fit a new parameter, β_2 to represent the "slope" from the X_2 predictor.

Write the new equations for $\hat{Y}$ in matrix form, with $\vec{\beta} = \begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{pmatrix}$.

🍓 Answer:

1.2.2

$$\hat{Y} = \begin{bmatrix} 1 & | & | \\ 1 & \vec{x}_1 & \vec{x}_2 \\ 1 & | & | \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{bmatrix}$$

★ 1.2.3 Write new equations for $\vec{e}$, the vector of residuals in matrix form, with $\vec{\beta} = \begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{pmatrix}$.

1.2.3

$$\vec{e}(\vec{\beta}) = y - \hat{y} = y - x\vec{\beta}$$

$$= \begin{bmatrix} 1 \\ y \\ 1 \end{bmatrix} - \begin{bmatrix} 1 & \vec{x}_1 & \vec{x}_2 \\ 1 & \vec{x}_1 & \vec{x}_2 \\ 1 & \vec{x}_1 & \vec{x}_2 \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{bmatrix}$$

Section 2: Brain and Body Weights

2.1: Loading the data

Use the provided code block to upload the file to collab.

```
In [758... #commented below out as this was done in VSCode
# load local file "Lab01_Data_01_brains.txt"
#from google.colab import files

#uploaded = files.upload()

#for fn in uploaded.keys():
#    print('User uploaded file "{name}" with length {length} bytes'.format(
#        #name=fn, length=len(uploaded[fn])))
```

? 2.1.1 Write code to read in your uploaded file as a pandas dataframe and visualize the first 5 rows.

```
In [759... # import numpy and pandas to prep your workspace
import numpy as np
import pandas as pd

## ? YOUR CODE HERE
# Read data in as a pandas df
df = pd.read_csv('Lab01_Data_01_brains.txt', sep='\t')

# visualize first five rows using data.head()
df.head(5)
```


Out [759...

	Index	Body.Weight	Brain.Weight
0	1	3.385	44.5
1	2	0.480	15.5
2	3	1.350	8.1
3	4	465.000	423.0
4	5	36.330	119.5

? 2.1.2 Identify and describe the variables in this dataframe. Are these categorical? Ordinal? Integer? Logical?

✓ Answer: There are 3 variables, namely, index, body weight, and brain weight. Index is a categorical variable (not integer) because even though it consists of whole numbers, it acts as a unique identifier or label rather than a measurable quantity or count. Both brain and body weight are continuous variables.

2.2: Data Exploration & Visualization

? 2.2.1 Let's predict brain weight from body weight. Visualize brain weight and body weight on a plot, with the independent variable on the x-axis. Be sure to label all parts of your figure (axis labels with units and title).

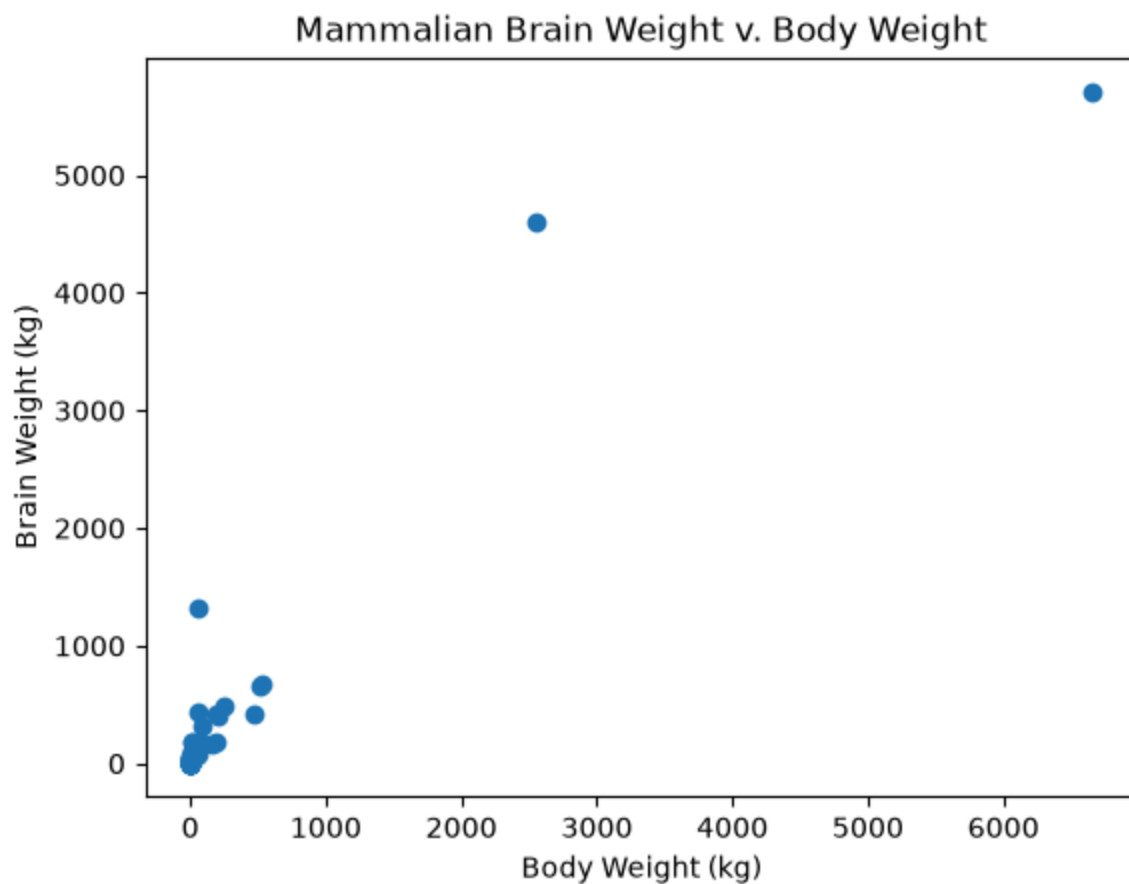
In [760...

```
# first import matplotlib
from matplotlib import pyplot as plt

## ? YOUR CODE HERE (to plot)
plt.figure()
plt.scatter(df["Body.Weight"], df["Brain.Weight"])
plt.xlabel("Body Weight (kg)")
plt.ylabel("Brain Weight (kg)")
plt.title("Mammalian Brain Weight v. Body Weight")
```

Out[760...

Text(0.5, 1.0, 'Mammalian Brain Weight v. Body Weight')



? 2.2.2 What are the assumptions for linear regression? Describe each assumption with <1 sentence.

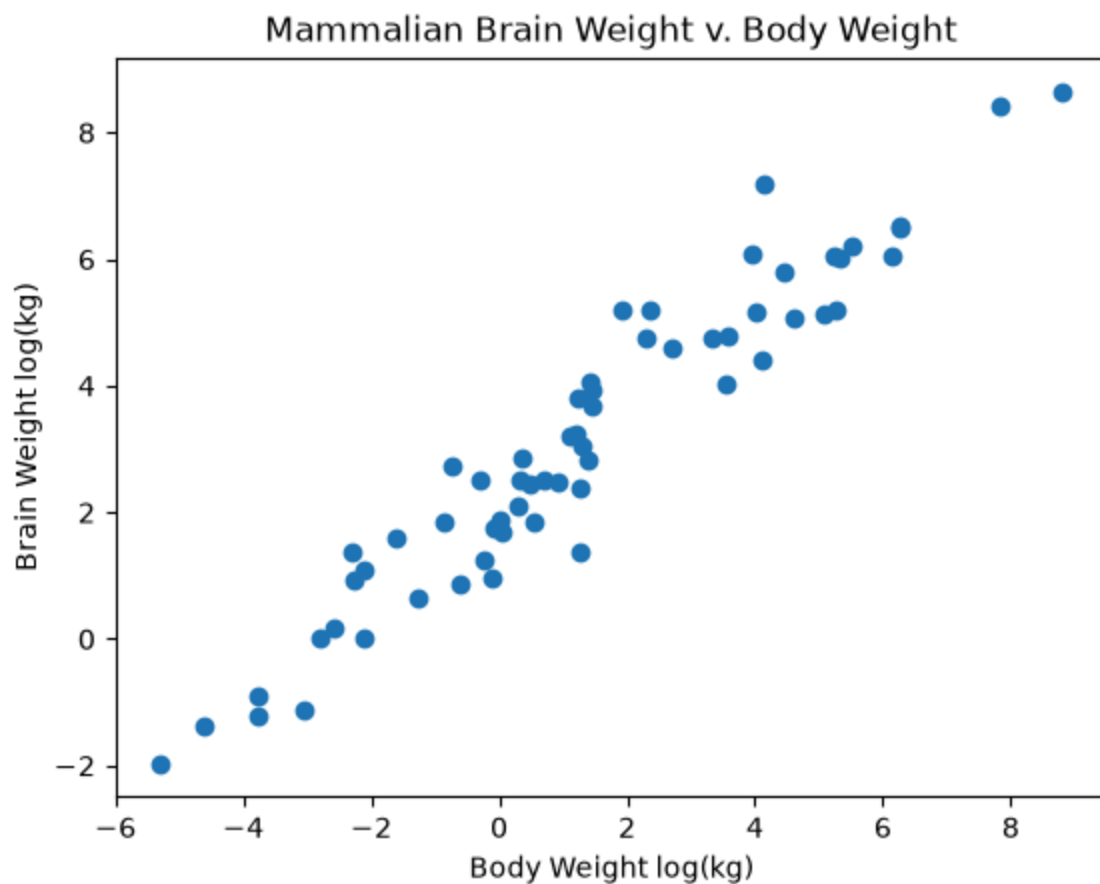
✓ Answer: LINE! There is a linear relationship between x and y. The residuals are independent and normally distributed. For any value of x, equal variance of the residuals.

2.3: Transformation

? 2.3.1 Transform the data to try to correct the linearity assumption and replot. Label your plot appropriately (include axis labels, title, units).

```
In [761... ## ? YOUR CODE HERE
# create new columns of log-scaled values in your pd df using data.assign()
logScaled = df.assign(logBody = np.log(df["Body.Weight"]), logBrain = np.log(df["Brain.W
# visualize the first 5 rows of the new df
logScaled.head(5)
# visualize the transformed data
plt.figure()
plt.scatter(logScaled["logBody"], logScaled["logBrain"])
plt.xlabel("Body Weight log(kg)")
plt.ylabel("Brain Weight log(kg)")
plt.title("Mammalian Brain Weight v. Body Weight")
```

```
Out[761... Text(0.5, 1.0, 'Mammalian Brain Weight v. Body Weight')
```



? 2.3.2 Have we satisfied these assumptions for the relationship between brain and body weight? What assumption(s) did the transformation help satisfy? *Your plot should qualitatively answer this question, you don't need to perform statistical tests or analysis beyond visual inspection.*

✓ Answer: Yes, visually, there is a linear relationship between x and y. Additionally, the errors seem to be independent & normally distributed and equal variance of the residuals for each x value.

2.4: A Simple Linear Model

? 2.4.1 Calculate the intercept and slope of a line of best fit on the transformed data using the formulas you derived above (also in the slides from class!)

In [762...

```
## ? YOUR CODE HERE

# calculate slope
xAvg = np.mean(logScaled["logBody"])
yAvg = np.mean(logScaled["logBrain"])
num = np.sum((logScaled["logBody"] - xAvg) * (logScaled["logBrain"] - yAvg))
denom = np.sum((logScaled["logBody"] - xAvg) ** 2)
m = num / denom
# be nice to your TAs
# try to print outputs from code labelled and easy to read
# this is an example if you name your slope variable m
print("Slope: ", m)
```

Slope: 0.7516859362419007

In [763...

```
## ? YOUR CODE HERE
# calculate and display intercept
b = yAvg - (m * xAvg)
```

```
# display and be nice to the TAs
print("Intercept: ", b)
```

Intercept: 2.134788676764636

2.5: Assessing the Model

Visualizing Error

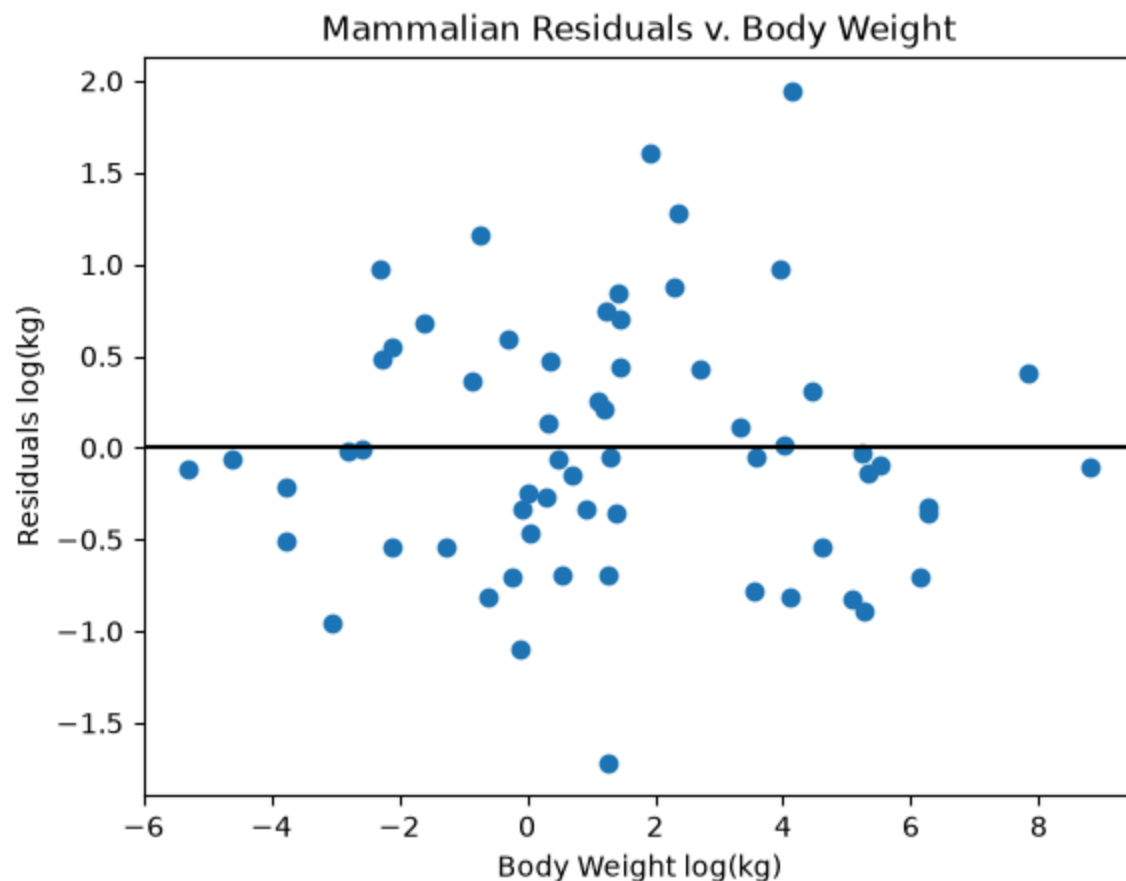
To do linear regression, we assume that there is a linear relationship between x and y , and we assume that the errors in y are normally and equally distributed at each x value. One of the easiest ways to visualize whether these assumptions hold in our model is to make a residuals plot, which displays the errors of the model (the residuals, $y - \hat{y}$) against the x -values themselves.

? 2.5.1 Calculate residuals and add them to your pandas dataframe. Make a residuals plot and label appropriately (include axis labels, units, plot title)

```
In [764... ## ? YOUR CODE HERE
# calculate residuals for each observation
yPred = m * logScaled["logBody"] + b
res = logScaled["logBrain"] - yPred

# Make a residuals plot using matplotlib
plt.figure()
plt.scatter(logScaled["logBody"], res)
plt.xlabel("Body Weight log(kg)")
plt.ylabel("Residuals log(kg)")
plt.title("Mammalian Residuals v. Body Weight")
plt.axhline(0, linestyle = "-", color = "black")
```

Out[764... <matplotlib.lines.Line2D at 0x161823d90>



? 2.5.2 Briefly interpret your residuals plot. Do you see any patterns? What do the patterns (or lack thereof) imply about your model?

✓ Answer: The random pattern of the residual plot is supportive of our previous assumption that the data is independent and normally distributed.

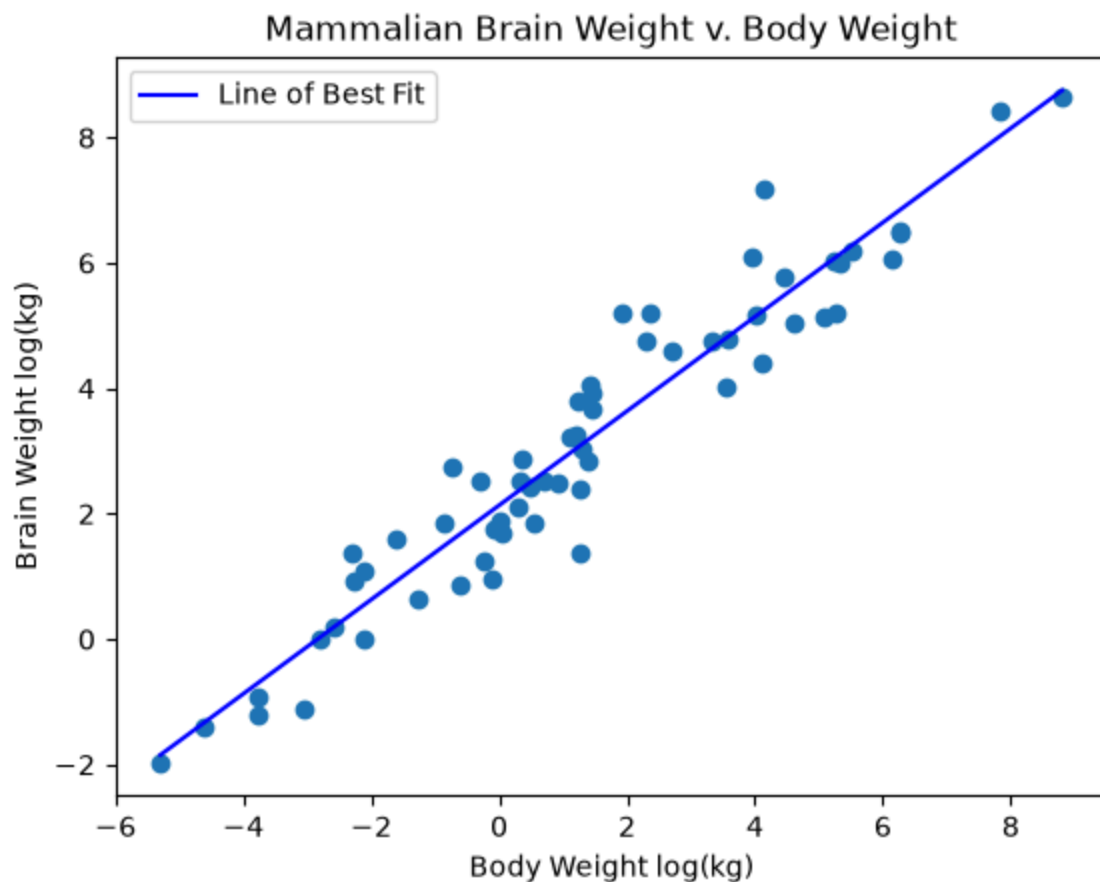
2.6 Visualize Model

? 2.6.1 Add a line of best fit to your log-scaled data plot from earlier.

```
In [765... ## ? YOUR CODE HERE
# visualize the transformed data
plt.figure()
plt.scatter(logScaled["logBody"], logScaled["logBrain"])
plt.xlabel("Body Weight log(kg)")
plt.ylabel("Brain Weight log(kg)")
plt.title("Mammalian Brain Weight v. Body Weight")
x = np.linspace(min(logScaled["logBody"]), max(logScaled["logBody"]), 100)
plt.plot(x, m*x + b, color="blue", label="Line of Best Fit")

# remember to add a legend
plt.legend()
```

Out[765... <matplotlib.legend.Legend at 0x161734cd0>



★ 2.7 Bonus - Quantitative Error Assessments

Please do this *after* finishing the required assignment.

★ 2.7.1 Calculate R^2 using the equation from class.

##🌟 YOUR CODE HERE

calculate slope

xAvgOG = np.mean(df["Body.Weight"])

yAvgOG = np.mean(df["Brain.Weight"])

numOG = np.sum((df["Body.Weight"] - xAvgOG) * (df["Brain.Weight"] - yAvgOG))

denomOG = np.sum((df["Body.Weight"] - xAvgOG) ** 2)

mOG = numOG / denomOG

calculate y-intercept

bOG = yAvgOG - mOG * xAvgOG

calculate residuals for each observation

yPredOG = mOG * df["Body.Weight"] + bOG

resOG = df["Brain.Weight"] - yPredOG

Calculate and display R2

SSres = np.sum(resOG ** 2)

SStot = np.sum((np.mean(df["Brain.Weight"]) - df["Brain.Weight"]) ** 2)

R2 = 1 - SSres / SStot

print("Coefficient of determination: ", R2,

"\nSSres: ", SSres,

"\nSStot: ", SStot)

Calculate and display R2 for the log model

SSres_ln = np.sum(res ** 2)

SStot_ln = np.sum((np.mean(logScaled["logBrain"]) - logScaled["logBrain"]) ** 2)

R2_ln = 1 - SSres_ln / SStot_ln

print("Coefficient of determination for the natural log model: ", R2_ln,

"\nSSres: ", SSres_ln,

"\nSStot: ", SStot_ln)

Coefficient of determination: 0.8726620843043331

SSres: 6722239.055504983

SStot: 52790553.534509696

Coefficient of determination for the natural log model: 0.9207837169027255

SSres: 28.92271042146065

SStot: 365.11067283912723

🌟 2.7.2 Explain what you calculated and how it connects to the interpretation of R^2 . (What's the total sum of squared errors? Why do we need to use it?)

🍌 : I calculated the coefficient of determination, R^2 , as 1 minus the ratio of the residual sum of squares (SSres) to the total sum of squares (SStot). SSres measures the squared deviation between each predicted value and its actual value, summed across all observations, the values are squared so that positive and negative residuals don't cancel each other out when summed. SStot measures the squared deviation between each actual y-value and the overall mean of y, representing the total variability in the data if you ignored the model entirely and just predicted the mean brain weight every time.

SStot matters because SSres alone isn't interpretable on its own, its magnitude depends on the scale of the data. SStot gives a baseline to compare against / the error of the simplest possible model with no predictors. R^2 expresses SSres relative to that baseline, which is what makes it comparable across models and datasets. A value close to 1 means SSres is small relative to SStot and a value close to 0 means the model does no better than that baseline.

🌟 2.7.3 Does it matter if we calculate R^2 from the log-scaled data or from the raw (untransformed) data of body/brain weights? Re-calculate R^2 from the untransformed data and estimates of brain weight from your log model and display it. Does this estimate match what you calculated above? Why or why not?

Note: You would need to compute R^2 in the space of untransformed data

In [767...

```
##★ YOUR CODE HERE
# recalculate some of the quantities you found above (SSres, SStot, R^2)
# and compare to what you had before
resNew = np.exp(logScaled["logBrain"]) - np.exp(yPred)
SSresNew = np.sum(resNew ** 2)
SStotNew = np.sum((np.mean(np.exp(logScaled["logBrain"]))) - np.exp(logScaled["logBrain"])) ** 2)
R2New = 1 - SSresNew / SStotNew
print("Transformed raw coefficient of determination: ", R2New,
      "Transformed raw r\nSSres: ", SSresNew,
      "Transformed raw\nSStot: ", SStotNew)

# doing the same for the natural log model
resNew_ln = logScaled["logBrain"] - yPred
SSresNew_ln = np.sum(resNew_ln ** 2)
SStotNew_ln = np.sum((np.mean(logScaled["logBrain"]) - logScaled["logBrain"]) ** 2)
R2New_ln = 1 - SSresNew_ln / SStotNew_ln
print("Log-space coefficient of determination: ", R2New_ln,
      "\nSSres: ", SSresNew_ln,
      "\nSStot: ", SStotNew_ln)
```

```
Transformed raw coefficient of determination: 0.9127159764899991 Transformed raw r
SSres: 4607771.915812099 Transformed raw
SStot: 52790553.53450962
Log-space coefficient of determination: 0.9207837169027255
SSres: 28.92271042146065
SStot: 365.11067283912723
```

🍓 : Yes, it matters which space you calculate R^2 in. Re-calculating R^2 in the untransformed (raw) space, using the log model's back-transformed predictions ($\hat{y}_{\text{raw}} = \exp(\hat{y}_{\text{log}})$) compared against the actual raw brain weights, gives: $SS_{\text{res}} = 4,607,771.92$, $SS_{\text{tot}} = 52,790,553.53$, and $R^2 = 0.9127$.

This does not match the $R^2 = 0.9208$ calculated in log space in 2.7.1, even though both numbers describe the same underlying model. The discrepancy exists because R^2 is not invariant under nonlinear transformations of the response variable. A small, roughly constant residual in log space becomes a multiplicative error in raw space, so the same log-space error translates into a tiny absolute deviation for a small-bodied animal but a very large absolute deviation for the largest animals in the dataset. Since SS_{res} in raw space is dominated by squared absolute errors, those large-animal residuals disproportionately inflate SS_{res} once back-transformed, pulling R^2 down relative to its log-space value.

SS_{tot} itself doesn't change between spaces in a way that's model-dependent because it only depends on the raw y-values and their mean.

★ 2.7.4 Calculate MSE and RMSE explicitly and display their values.

In [768...

```
##★YOUR CODE HERE
# calculate MSE and RMSE
MSE = np.sum(resOG ** 2) / len(resOG)
RMSE = np.sqrt(MSE)

# display MSE and RMSE
print("MSE: ", MSE, "\nRMSE: ", RMSE)

# doing the same for ln
# calculate MSE and RMSE
```

```
MSE_ln = np.sum(res ** 2) / len(res)
RMSE_ln = np.sqrt(MSE_ln)
# display MSE and RMSE
print("MSE for natural log model: ", MSE_ln, "\nRMSE for natural log model: ", RMSE_ln)
```

```
MSE: 108423.21057266102
RMSE: 329.27679932339754
MSE for natural log model: 0.4664953293783976
RMSE for natural log model: 0.6830046334970192
```

2.8: Model Interpretation

? 2.8.1 This model was built on data from 62 mammalian species. Does it do a good job predicting species brain weight from body weight? Suggest one way to make this model better.

✓ Answer: The model does a reasonably good job overall. R^2 of 0.92 in log space means it explains most of the variance in brain weight across the 62 species, but its typical prediction is still off by roughly a factor of 2 (RMSE of about 0.68 in log space, which back-transforms to an about 2-fold multiplicative error), so it's far from precise at the level of an individual species. This imprecision reflects that the model treats all mammals as following one universal brain-body scaling. For example, primates, have disproportionately large brains for their body size relative to some other mammals, so a single pooled slope masks this nuance. To improve this model, we could add taxonomic order/class as a categorical predictor which would help the model account for lineage-specific scaling instead.

? 2.8.2 Is this model a good way to predict human brain weights from body weights?

✓ Answer: No, because this model predicts species-average brain weight from species-average body weight across 62 mammals, but humans don't necessarily follow general mammalian brain-body scaling.

Section 3: Regularization Calculations

In this section, we will be performing calculations with and without regularizers to understand how regularization affects weights, loss, and the overall network. The calculations will be performed on ordinary least squares regression networks used in Section 1 if you need to refer back.

Please round to **two decimal places**.

The formula for an OLS regression will be represented as $\hat{y} = w_1x_1 + w_2x_2$

3.1 Calculate SSE

? Given three datapoints: $(x_1 = 1, x_2 = 2, y = 4)$, $(x_1 = 2, x_2 = 0, y = 3)$, $(x_1 = 0, x_2 = 1, y = 2)$. In Section 1, we defined error functions for general OLS. Here we will be using a sum of squares error (SSE) function expressed as $SSE = \sum_{i=1}^n (y_i - \hat{y}_i)^2$. MSE differs from SSE in that MSE divides by the number of terms (N), but they both produce a square error. Given the SSE of the OLS with the given datapoints, find w_1 and w_2 .

Hint: Find the partial derivatives of w_1 and w_2 (like in previous sections) and then solve the system of equations to find w_1 and w_2

You may do this math on paper and insert a picture of your work here.

✓ Answer:

$$3.1 \quad SSE = \sum_{i=1}^3 (y_i - \hat{y}_i)^2 \quad \hat{y} = \omega_1 x_1 + \omega_2 x_2$$

$$\begin{aligned} (1, 2, 4) \\ \hat{y}_1 &= \omega_1(1) + \omega_2(2) \\ &= \omega_1 + 2\omega_2 \end{aligned}$$

$$\begin{aligned} (2, 0, 3) \\ \hat{y}_2 &= \omega_1(2) + \omega_2(0) \\ &= 2\omega_1 \end{aligned}$$

$$\begin{aligned} (0, 1, 2) \\ \hat{y}_3 &= \omega_1(0) + \omega_2(1) \\ &= \omega_2 \end{aligned}$$

$$SSE(\omega_1, \omega_2) = (4 - \omega_1 - 2\omega_2)^2 + (3 - 2\omega_1)^2 + (2 - \omega_2)^2$$

$$\begin{aligned} \frac{\partial SSE}{\partial \omega_1} &= 0 = -2(4 - \omega_1 - 2\omega_2) - 4(3 - 2\omega_1) \\ &= -8 + 2\omega_1 + 4\omega_2 - 12 + 8\omega_1 \\ &= 10\omega_1 + 4\omega_2 - 20 \end{aligned}$$

$$\begin{aligned} \frac{\partial SSE}{\partial \omega_2} &= 0 = -4(4 - \omega_1 - 2\omega_2) - 2(2 - \omega_2) \\ &= -16 + 4\omega_1 + 8\omega_2 - 4 + 2\omega_2 \\ &= 10\omega_2 + 4\omega_1 - 20 \end{aligned}$$

$$\begin{aligned} 10\omega_1 + 4\omega_2 - 20 &= 10\omega_2 + 4\omega_1 - 20 \\ 10\omega_1 + 4\omega_2 &= 10\omega_2 + 4\omega_1 \\ 6\omega_1 &= 6\omega_2 \\ \omega_1 &= \omega_2 \end{aligned}$$

$$\begin{aligned} 0 &= 10\omega_1 + 4\omega_2 - 20 \\ &= 10\omega_1 + 4(\omega_1) - 20 \\ &= 14\omega_1 - 20 \end{aligned}$$

$$\begin{aligned} 20 &= 14\omega_1 \\ \frac{10}{7} &= \omega_1 = \omega_2 \end{aligned}$$

$$\begin{aligned} (1, 2, 4) \\ \hat{y}_1 &= \omega_1(1) + \omega_2(2) \\ &= \omega_1 + 2\omega_2 \\ &= \frac{30}{7} \end{aligned}$$

$$\begin{aligned} (2, 0, 3) \\ \hat{y}_2 &= \omega_1(2) + \omega_2(0) \\ &= 2\omega_1 \\ &= \frac{20}{7} \end{aligned}$$

$$\begin{aligned} (0, 1, 2) \\ \hat{y}_3 &= \omega_1(0) + \omega_2(1) \\ &= \omega_2 \\ &= \frac{10}{7} \end{aligned}$$

$$y_1 = 4 = \frac{28}{7}$$

$$y_2 = 3 = \frac{21}{7}$$

$$y_3 = 2 = \frac{14}{7}$$

$$\frac{28}{7} - \frac{30}{7} = -\frac{2}{7}$$

$$\frac{21}{7} - \frac{20}{7} = \frac{1}{7}$$

$$\frac{14}{7} - \frac{10}{7} = \frac{4}{7}$$

$$\begin{aligned} SSE &= \left(-\frac{2}{7}\right)^2 + \left(\frac{1}{7}\right)^2 + \left(\frac{4}{7}\right)^2 \\ &= \frac{4 + 1 + 16}{49} \\ &= \frac{21}{49} \\ &= \frac{3}{7} \end{aligned}$$

3.2 Calculation of Lasso and Ridge Regression

? For $\lambda = 1$, calculate the SSE with no regularizer, SSE with lasso regularization (L1), and SSE with ridge regression (L2). You should be reporting 3 answers.

Hint: Remember no regularizer is just SSE, lasso is $\lambda(w_1 + w_2)$, and ridge is $\lambda(w_1^2 + w_2^2)$.

✓ Answer:

3.2

No regularizer: $\frac{3}{7}$

$$\begin{aligned}\text{Lasso (L1): } SSE + \lambda(w_1 + w_2) \\ \frac{3}{7} + \frac{20}{7} \\ = \frac{23}{7}\end{aligned}$$

$$\begin{aligned}\text{Ridge (L2): } SSE + \lambda(w_1^2 + w_2^2) \\ \frac{3}{7} + \left(\frac{100}{49} + \frac{100}{49}\right) \\ = \frac{21}{49} + \frac{200}{49} \\ = \frac{221}{49}\end{aligned}$$

3.3 Understanding Regularizer Penalty

? What would happen to the loss as λ changes? What does this mean for the weights and overall loss?

✓ Answer: As λ increases, the penalty on large weights grows, so the model shrinks the weights to reduce that penalty (with lasso, some weights can go to zero); as λ decreases toward 0, the weights drift back toward the unregularized OLS solution. Because OLS weights are the minimizer of SSE alone, moving away from them to satisfy a penalty can only increase training SSE. Smaller λ gets closer to the OLS fit but risks overfitting, so λ is trading off training fit against generalization.

3.4 Understanding Ridge and Lasso

? Is the loss for the L1 or L2 regularizer greater? In general, how does L1 vs L2 regularization affect the weights?

✓ Answer: Here, L2 (ridge) loss is greater. In general, L1 can zero weights out (sparse, does feature selection) and L2 shrinks weights toward zero but keeps all of them nonzero (dense, but smaller). If $|w| > 1$, squaring makes it bigger and leads to L2 penalty $>$ L1 penalty. If $|w| < 1$, squaring makes it smaller and leads to L2 penalty $<$ L1 penalty.

Section 4: Multiple Linear Regression

4.1: Load the data, prep for modelling

Run the following codeblock to upload the file `data_01_wdbc.data.csv`

In [769...

```
#using VSCode, not CoLab
# upload local file "data01_breastcancer.csv"
#uploaded = files.upload()

#for fn in uploaded.keys():
#    print('User uploaded file "{name}" with length {length} bytes'.format(
#        name=fn, length=len(uploaded[fn])))

# load file
# be sure to specify the header explicitly for this data, as it isn't included in the fi
colnames = ["ID", "diagnosis",
            "radius1", "texture1", "perimeter1", "area1", "smoothness1", "compactness1", "conc
            "concave_points1", "symmetry1", "fractal_dimension1",
            "radius2", "texture2", "perimeter2", "area2", "smoothness2", "compactness2", "conc
            "concave_points2", "symmetry2", "fractal_dimension2",
            "radius3", "texture3", "perimeter3", "area3", "smoothness3", "compactness3", "conc
            "concave_points3", "symmetry3", "fractal_dimension3"]
```

? 4.1.1 Load the uploaded `.csv` file with breast cancer information as a pandas dataframe, as before. Check that you've loaded the data correctly and specified the header correctly by visualizing the first 5 rows. You should have 569 observations of 32 features.

In [770...

```
## ? YOUR CODE HERE
# read in the uploaded file as a pandas df, specify the colnames vector above as the col
```

note that there is no header in data_01_wdbc.data.csv file

```
csv_name = "data_01_wdbc.data.csv"
cancer_data = pd.read_csv(csv_name, names=colnames)
# visualize the first 5 rows to check your work
cancer_data.head(5)
```

Out[770...

	ID	diagnosis	radius1	texture1	perimeter1	area1	smoothness1	compactness1	concav
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1

5 rows x 32 columns

? 4.1.2 Split your data into predictors and output variable. You'll need to create a dummy variable of 0s and 1s to code for "benign" or "malignant" samples, respectively. Make sure to drop columns you don't want to use as predictors from the predictor dataframe (e.g. ID probably shouldn't be used as a predictor variable).

In [771...

```
## ? YOUR CODE HERE
# Create a dummy variable for output (tumor diagnosis)
y = np.where(cancer_data["diagnosis"] == "M", 1, 0)
# quick check that nothing was mapped to NaN
check = np.sum(y)
print("Check: ")
print(check)
## Create separate X dataframe for predictors
X = cancer_data.iloc[:, 2:]
print(y.shape)
print(X.shape)
```

Check:

212

(569,)

(569, 30)

? 4.1.3 Split the data randomly into a train and test set; we will compare error rates and metrics on the test data to contrast models. Use `random_state = 42` to make this split reproducible across the class. Call your new predictor dataframes `X_train` and `X_test` for compatibility with the TA scaling code below.

In [772...

```
import sklearn

# load train_test_split for this task
from sklearn.model_selection import train_test_split
## ? YOUR CODE HERE
XTrain, XTest, yTrain, yTest = train_test_split(X,
                                                y,
                                                test_size = 0.2,
                                                random_state = 42)

print(XTrain.shape)
print(XTest.shape)
```

```
print(yTrain.shape)
print(yTest.shape)
```

```
# reproducibly split the data into 80% training and 20% testing
```

```
(455, 30)
(114, 30)
(455,)
(114,)
```

Standardize and scale your predictor variables before building models.

Hint: Should the entire dataset be included in the scaling calculation?

In [773...

```
# load standard scaler to scale data
from sklearn.preprocessing import StandardScaler

## ? YOUR CODE HERE
# select a scaler
scaler = StandardScaler()

# fit and transform the training data
XTrainScaled = scaler.fit_transform(XTrain)
XTestScaled = scaler.transform(XTest)

# apply the same transformation to the test data - use the parameters
print(XTrainScaled.mean())
print(XTrainScaled.std())
```

```
3.839012949619589e-18
1.0
```

? 4.1.4 Are we standardizing on just entire dataset or just one specific part? If so, why does this occur?

✓ Answer: We standardize using only the training data. We do this by fitting the scaler (mean/std) on XTrain, then applying that same transformation to XTest without refitting. This matters because fitting the scaler on the full dataset would let information about the test set affect the values used for training. This would make test performance look better than it actually is.

4.2: Logistic Regression Model Fitting

? 4.2.1 Fit a logistic regression model using all the available continuous predictors to predict whether the tissue is benign or malignant. Be sure *not* to add regularization - the [documentation](#) makes it clear that L2 regularization is the default choice.

In [774...

```
import sklearn
print(sklearn.__version__)
```

```
1.9.0
```

In [775...

```
# load the model I want to use
from sklearn.linear_model import LogisticRegression

## ? YOUR CODE HERE
# Fitting a logistic regression model
logReg = LogisticRegression(penalty = None)
logReg.fit(XTrainScaled, yTrain)
```

```
/Users/saanvee/Desktop/BME 304/BME-304/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1403: FutureWarning: 'penalty' was deprecated in version 1.8 and will be removed in 1.10. To avoid this warning, leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', l1_ratio set to a float between 0 and 1 instead of penalty='elasticnet', and C=np.inf instead of penalty=None.
warnings.warn(
```

Out[775]...

▼ LogisticRegression ⓘ ?

- Parameters
- Fitted attributes

? 4.2.2 Fit a second model, this time with L2 regularization.

In [776]...

```
## ? YOUR CODE HERE

# use L2 regularization to build a model
l2Reg = LogisticRegression(penalty = "l2")
l2Reg.fit(XTrainScaled, yTrain)
```

```
/Users/saanvee/Desktop/BME 304/BME-304/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1403: FutureWarning: 'penalty' was deprecated in version 1.8 and will be removed in 1.10. To avoid this warning, leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', l1_ratio set to a float between 0 and 1 instead of penalty='elasticnet', and C=np.inf instead of penalty=None.
warnings.warn(
```

Out[776]...

▼ LogisticRegression ⓘ ?

- Parameters
- Fitted attributes

4.3: Evaluating the Models

? 4.3.1 Use both models to make predictions on *both* the training and test data. Report the accuracy for each model on both training and test data.

In [777]...

```
# load evaluation metrics
from sklearn.metrics import accuracy_score

## ? YOUR CODE HERE
## Make predictions on the test data, using all 3 models
yTrainPred = logReg.predict(XTrainScaled)
yTestPred = logReg.predict(XTestScaled)
yTrainPredl2 = l2Reg.predict(XTrainScaled)
yTestPredl2 = l2Reg.predict(XTestScaled)

# calculate accuracy for each set of predictions
noRegTrain = accuracy_score(yTrain, yTrainPred)
noRegTest = accuracy_score(yTest, yTestPred)
l2Train = accuracy_score(yTrain, yTrainPredl2)
```



```
l2Test = accuracy_score(yTest, yTestPredl2)
```

```
## display accuracy values
#print("Accuracies of full, and l2 models on test data:\n", accuracies,
#      "\nAccuracies of full, and l2 models on training data:\n", accuracies_train)
print("For No Regularization:")
print(f"Training data: {noRegTrain}")
print(f"Test data: {noRegTest}")
print("")
print("For L2-Regularization:")
print(f"Training data: {l2Train}")
print(f"Test data: {l2Test}")
```

For No Regularization:
Training data: 1.0
Test data: 0.9385964912280702

For L2-Regularization:
Training data: 0.9868131868131869
Test data: 0.9736842105263158

? 4.3.2 Which model has the highest accuracy on the *training* data? Is this what you expect? Why or why not? (1 sentence).

✓ Answer: The no-regularization model has the highest training accuracy (1.0, perfect), which is expected because with no penalty constraining the weights, the model has maximum freedom to minimize training error, so it will always match or exceed a regularized model on the data it was fit to.

? 4.3.3 Compare and discuss the accuracies on the *test* data. Consider overfitting.

✓ Answer: The L2-regularized model actually has higher test accuracy (0.974) than the unregularized model (0.939), despite having slightly lower training accuracy (0.987 vs. 1.0). The unregularized model's perfect training fit didn't transfer, indicative of overfitting. This shows the penalty successfully traded a small amount of training fit for better generalization.

★ Bonus - 4.4: Evaluating Models with ROC Curves

Since this is a logistic regression model that actually outputs probabilities we threshold into classifications, we can sketch out a ROC curve for each model.

★ 4.4.1 Make a ROC curve for each model using the testing data and calculate the AUC for each. For the purposes of this assignment, also make a ROC curve for each model on the training data, not just the testing data. Make sure to label your axes appropriately.

Hint: We've asked for several different ROC curves and their AUC! You could write a function to print a ROC curve and display the AUC, given a model and data (training or test X and y values). Skeleton code is provided below.

```
In [778... # load necessary evaluation metrics
from sklearn.metrics import roc_auc_score, confusion_matrix, roc_curve, auc

## ? YOUR CODE HERE
## Write function to make a ROC curve and calculate AUC
def plot_my_roc_curve(model, X_data, y_values):
    # write your code here
    # predict probabilities of the positive class (malignant tumor)
```

```

y_probs = model.predict_proba(X_data)[: , 1]

# Compute ROC curve and AUC
fpr, tpr, thresholds = roc_curve(y_values, y_probs)
roc_auc = auc(fpr, tpr)

## plotting
plt.figure(figsize = (4,4))
# Display ROC curve and AUC
plt.plot(fpr, tpr, label=f"ROC curve (AUC = {roc_auc:.3f})")

# add a line for where random chance is on the ROC curve
plt.plot([0, 1], [0, 1], linestyle="--", color="gray", label="Chance")

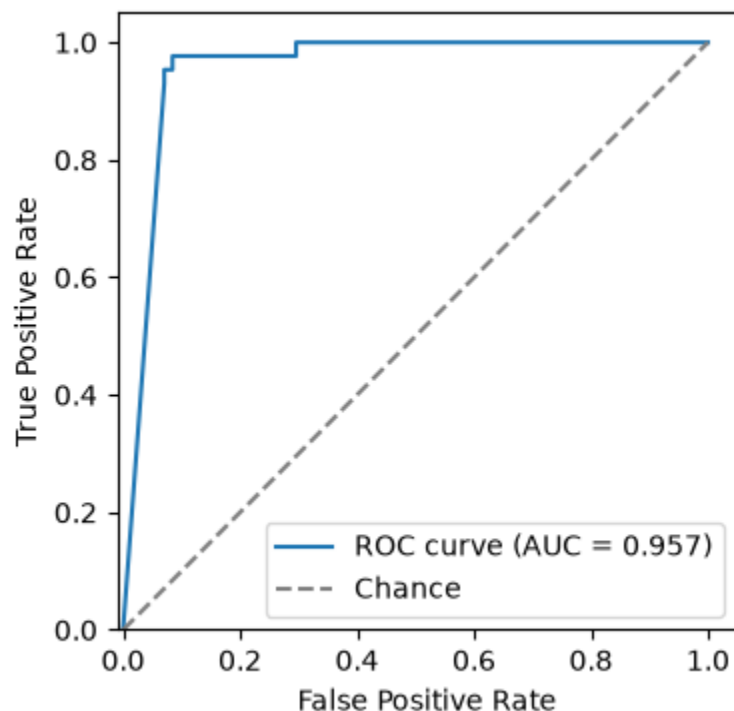
# change the limits of the x and y axis so you can see the full plot
plt.xlim([-0.01, 1.05])
plt.ylim([0.0, 1.05])
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend(loc="lower right")
plt.show()

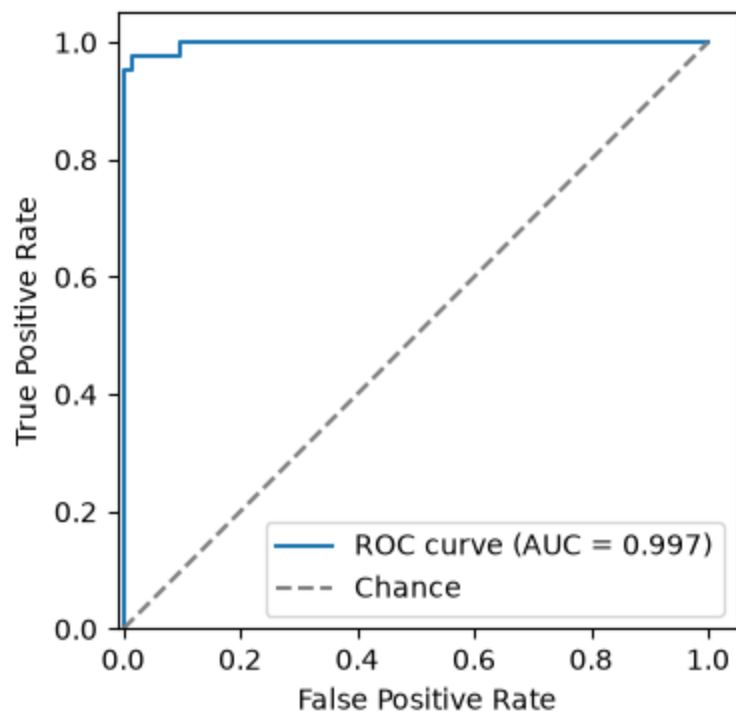
return roc_auc

# Apply this function to your test data
test_auc_noreg = plot_my_roc_curve(logReg, XTestScaled, yTest)
test_auc_l2 = plot_my_roc_curve(l2Reg, XTestScaled, yTest)
print(f"Test AUC no regularization: {test_auc_noreg}")
print(f"Test AUC with L2: {test_auc_l2}")

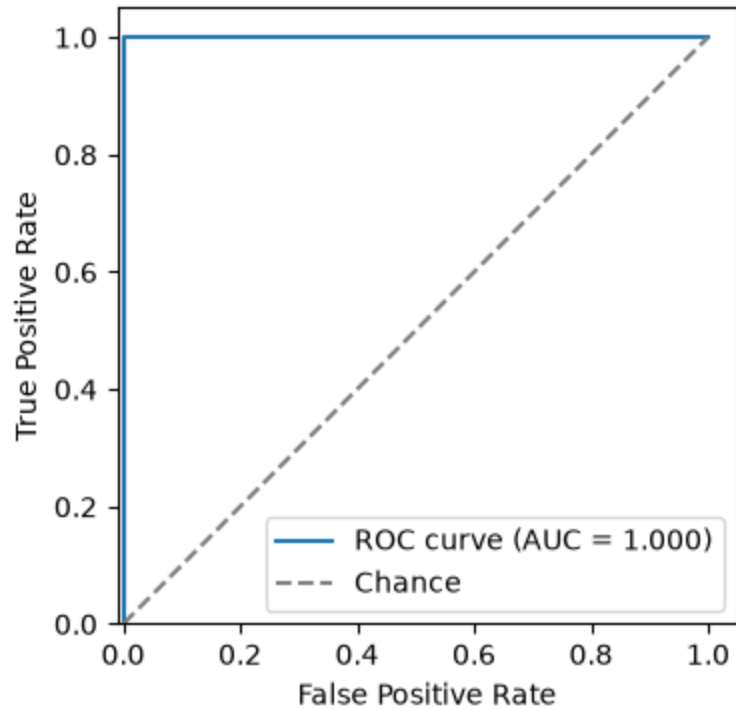
## Apply this function to your training data (unusual, but we are comparing in this assi
train_auc_noreg = plot_my_roc_curve(logReg, XTrainScaled, yTrain)
train_auc_l2 = plot_my_roc_curve(l2Reg, XTrainScaled, yTrain)
print(f"Train AUC no regularization: {train_auc_noreg}")
print(f"Train AUC with L2: {train_auc_l2}")

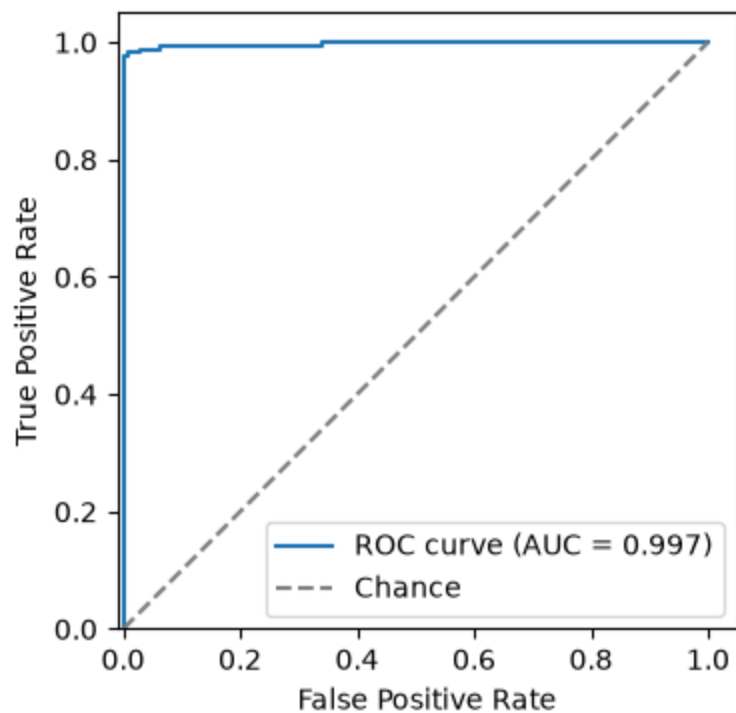
```





Test AUC no regularization: 0.9567638388470358
Test AUC with L2: 0.99737962659679





Train AUC no regularization: 1.0

Train AUC with L2: 0.9974138287747756

★ 4.4.2 Briefly compare and contrast your ROC curves and AUC on both models across training and testing data.

🍓 Answer: The unregularized model shows the clearest overfitting because its perfect training AUC (1.000) drops to 0.957 on test data. The L2 model, by contrast, is pretty stable across both with 0.997 on training and 0.997 on test, meaning there is almost no train-test gap at all. Notably, the L2 model's test AUC (0.997) actually beats the unregularized model's test AUC (0.957), so regularization isn't just reducing the train-test gap, it also produced a better-discriminating model.

Feedback

? What's one thing you learned in this assignment?

✅ Answer: I feel like I have a better understanding of MSE, RMSE, SSres, and SStot. Also, I learned how to embed images into a markdown box which took me a while to figure out.

? What do you wish had been different about this assignment?

✅ Answer: I wish the partial derivatives were omitted from the assignment. I appreciate how detailed the background notebook was, however, I wish all of that information was just integrated into the notebook that we are submitting.